Projet SR70, Gestionnaire de réunions

Aymeric DEGIEUX, Tiago DA SILVA RODRIGUES

1. Sujet et objectifs

Ce projet consiste à développer une application de gestion de réunions pour plusieurs équipes. Elle permet de consulter les plannings de deux salles et ceux des membres, de demander une réunion avec une durée, une heure de début souhaitée, une priorité, et sélectionner les participants puis de visualiser l'historique des opérations.

L'application doit tenir compte de la disponibilité de la salle et de celle de tous les participants. Les réunions sont de priorité **MINO*** ou **MAJOR**.

Une réunion majeure est prioritaire sur les réunions mineures déjà planifiées : si nécessaire, celles-ci sont déplacées vers un autre créneau ou annulées si aucun créneau de remplacement n'est disponible.

L'objectif est également de manipuler certains concepts abordés en cours, tel que les files de messages, la mémoire partagée, ...

2. Fonctionnalités réalisées

- Affichage du planning des deux salles, créneau par créneau, ainsi que des détails d'une réunion sélectionnée.

- Consultation des équipes et de leurs membres, puis affichage du planning d'un membre.

- Formulaire de demande permettant de choisir une priorité, une durée, un début précis ou automatique, et les participants.

- Recherche d'un créneau respectant la capacité de la salle et la disponibilité de chaque participant.

- Priorité des réunions majeures sur les réunions mineures : les réunions mineures bloquantes sont libérées, puis replacées si possible ou annulées.

- Suppression d'une réunion depuis l'interface.

- Envoi de notifications aux participants lors de l'acceptation, du refus, du déplacement ou de l'annulation d'une réunion.

- Consultation de l'historique des événements de réservation et de suppression.

L'interface présente six équipes. Le nombre de membres d'une équipe est initialisé entre cinq et dix. Deux salles sont configurées : la salle A, de capacité huit, et la salle B, de capacité quinze. La journée est représentée par dix créneaux horaires à partir de 8 h.

3. Architecture générale

Le programme est organisé autour de quatre rôles principaux :

1. **Le lanceur launcher.c** crée les ressources IPC, initialise les structures partagées et démarre les différents processus.

2. **Les processus member.c** représentent individuellement les membres des équipes, qui consultent périodiquement les messages qui leur sont destinés et affichent les notifications reçues (dans le terminal)

3. **Le gestionnaire.c** est la véritable partie métier du gestionnaire d'agenda, qui reçoit les demandes, applique les règles de réservation, met à jour les plannings et notifie les participants.

4. **L'interface graphique (ui.c)** affiche les plannings et l'historique, recueille les actions de l'utilisateur et transmet les demandes au gestionnaire. Elle est réalisée avec SDL2 et SDL_ttf.

Les structures de données communes sont définies dans commun.h. Elles décrivent par exemple des constantes tel que le nombre maximum d'équipe ou de membre par équipe, la taille des salles, différentes valeurs logiques, ou encore défini la structure des messages utilisé par les files de messages, ...

Les prototypes des fonctions des différents modules sont regroupés dans `modules.h`.

4. Déroulement d'une demande de réunion

1. L'utilisateur sélectionne un membre et renseigne les caractéristiques de la réunion dans l'interface.

2. L'interface construit une demande et l'envoie au gestionnaire dans la file de messages System V.

3. Le gestionnaire vérifie les paramètres, puis recherche un créneau compatible avec la salle et les agendas des participants.

4. Si la demande est acceptée, le gestionnaire met à jour les disponibilités partagées et envoie une notification à chaque participant. Dans le cas contraire, il envoie une notification de refus.

5. Pour une réunion majeure, les réunions mineures qui entrent en conflit sont retirées temporairement des plannings. Le gestionnaire tente ensuite de les replacer ; celles qui ne peuvent pas l'être sont annulées.

6. Les événements sont ajoutés à l'historique consultable dans l'interface.

5. Choix de conception

Un gestionnaire central

Le gestionnaire est le seul processus chargé de traiter les demandes de réservation et de modifier les plannings à la suite de ces demandes. Cette centralisation permet d'appliquer les règles de réservation au même endroit et de traiter les requêtes reçues les unes après les autres.

Une file de messages pour les échanges

Les commandes destinées au gestionnaire utilisent un type de message commun. Les notifications du gestionnaire sont envoyées avec un type correspondant à l'identifiant du membre destinataire. Le champ « requestType » distingue ensuite la nature de la commande ou de la notification.

Ce mécanisme sépare les échanges du partage des données : la file transporte les demandes et les notifications, tandis que les structures complètes des agendas restent en mémoire partagée.

Des agendas indexés par créneau

Les salles et les membres disposent d'un tableau de disponibilités indexé par heure. Chaque case indique soit qu'elle est libre (-1), soit l'identifiant de la réunion qui

l'occupe. Cette représentation facilite la détection des conflits sur toute la durée d'une réunion.

La recherche essaie les salles pour une même heure avant de passer à l'heure suivante. Ainsi, le moteur ne choisit pas automatiquement une heure plus tardive dans la salle A sans avoir d'abord vérifié si la salle B est libre à l'heure demandée.

Deux niveaux de priorité

Une réunion mineure est placée dans le premier créneau disponible à partir de l'heure souhaitée ; si aucune heure n'est précisée, la recherche commence au début de la journée. Une réunion majeure conserve la priorité sur les réunions mineures, mais ne déplace pas une autre réunion majeure.

6. Concepts de programmation système utilisés

Processus

Le lanceur utilise « fork() » pour créer des processus distincts pour les membres, le gestionnaire et l'interface. Chaque processus exécute une fonction dédiée, tandis que le processus parent (~ launcher.c) conserve la responsabilité du démarrage et de l'arrêt de l'application.

Communication interprocessus par file de messages

Une file System V est créée avec « msgget() ». Les messages sont envoyés avec « msgsnd() » et reçus avec « msgrcv() ». Le gestionnaire attend les commandes qui lui sont destinées, les membres vérifient périodiquement leurs notifications sans bloquer leur boucle (car de base devait pouvoir envoyer des requêtes pour de nouvelles réunion, même si maintenant cela est géré dans l'ui).

Mémoire partagée

Le lanceur crée un segment System V avec « shmget() » et le rattache à son espace mémoire avec « shmat() ». Les processus enfants héritent de l'accès à ce segment après leur création. Il contient les équipes, les membres, les salles, les réunions et l'historique. Le segment est détaché et supprimé à la fin de l'application.

Sémaphores et protection de la mémoire partagée

Le gestionnaire et l'interface graphique accèdent tous deux aux données de l'agenda : le premier les modifie lors du traitement des demandes, tandis que la seconde les consulte pour afficher les plannings et l'historique. Pour éviter qu'une lecture ait lieu pendant une modification, le lanceur crée un sémaphore System V initialisé à 1. Ce sémaphore agit comme un verrou : un processus doit le prendre avant d'accéder aux données partagées et le libérer après avoir terminé. Si le verrou est déjà utilisé, l'autre processus attend.

Signaux et arrêt des processus

À la fermeture de l'interface, le lanceur attend sa fin, puis envoie SIGTERM aux processus membres encore actifs. Le gestionnaire reçoit une commande STOP_MANAGER par la file de messages afin de terminer proprement sa boucle. Le lanceur récupère ensuite les processus restant avec « waitpid() » et « wait() », puis supprime les ressources IPC.

Notions non utilisées

Le projet n'utilise pas de threads. Le parallélisme repose sur des processus créés avec « fork() ». Les signaux servent simplement ici à demander l'arrêt des membres ; ils ne sont pas utilisés comme mécanisme de communication pour les demandes de réunion.

7. Difficultés rencontrées et solutions apportées

Une première difficulté concernait la recherche de salle. Une recherche qui parcourait d'abord toutes les heures de la salle A pouvait retenir un créneau plus tardif, sans vérifier si la salle B était libre à l'heure souhaitée. L'ordre de parcours a été organisé par heure puis par salle afin que toutes les salles soient examinées avant de décaler la réunion.

Une autre difficulté concernait les conflits entre priorités. Une réunion majeure demandée à une heure déjà occupée par une mineure ne doit pas simplement être placée après celle-ci. Le gestionnaire identifie désormais les réunions mineures qui bloquent la salle ou l'agenda d'un participant, libère leurs créneaux, réserve la réunion majeure, puis tente de replacer les mineures. Si une mineure ne peut pas être replanifiée, elle est annulée et ses participants sont notifiés.

Enfin, le projet nécessite de coordonner plusieurs processus qui doivent consulter les mêmes informations. La séparation entre les messages de commande et les données conservées en mémoire partagée permet de distinguer les requêtes à traiter de l'état courant des plannings.

8. Vérifications et démonstration

Les essais fonctionnels peuvent notamment couvrir les scénarios suivants :

- réserver une réunion dans la salle B à l'heure demandée lorsque la salle A est occupée

- demander une réunion majeure sur un créneau occupé par une réunion mineure et vérifier le déplacement ou l'annulation de cette dernière

- vérifier qu'une réunion majeure déjà planifiée n'est pas déplacée

- vérifier le refus d'une réunion lorsqu'aucun créneau n'est disponible

- supprimer une réunion depuis l’interface et vérifier la mise à jour des agendas et les notifications

9. Bilan et améliorations possibles

Le projet met en œuvre une application multiprocessus dans laquelle une interface graphique, un gestionnaire central et des processus membres communiquent au moyen d'IPC System V. Les principales règles de planification, la gestion des priorités et les notifications sont intégrées à l'application.

Les améliorations possibles comprennent une gestion plus complète des erreurs de création et d'arrêt des processus, ainsi que des tests automatisés du moteur de réservation. La portabilité de la configuration de la police SDL_ttf et la documentation de la procédure de compilation peuvent également être améliorées, car pour l’instant propre à uniquement un PC ayant comme OS Linux.